PoolUp: Demo and Viva Guide

Who shows what, in what order, and the questions to expect

Before the demo (5 minutes)

1. `cd server && npm run db:setup` **on the day**. The data is generated relative to the moment you seed, so the demo student always has fresh rides to rate, payments to confirm, debts to pay and requests to approve. The setup ends with `✓ All 13 cross-module integrity checks pass`.
2. `npm start` (serves the built UI on http://localhost:4000) or `npm run dev` here plus `npm run dev` in `client/`.
3. Log in as **Aarav Mehta,** `2023A7PS0001H` / `poolup123` (the login page has a "Fill in the demo login" button). Every seeded student has the same password; the account menu (top right) switches to any of them.
4. Keep a second tab on **Database** and a terminal with `psql` open.

The 15-minute team demo: one ride through five modules

#	Who	Time	Do this	Say this
1	M1	0:00	**Home**	"Everything waiting for Aarav comes from one query, `my_todo`, a UNION ALL over six sources from five modules. Click SQL on any card."
2	M1	0:40	**Offer a ride**: Main Gate → Airport with a stop; then try the same time again	"`fn_create_ride` writes the ride, its stops and Aarav's own seat atomically. The second one is refused by a trigger with a row lock: rule R10, no overlapping rides."
3	M1	1:30	**Find a ride**: Main Gate → Ameerpet	"Rides that *pass through* Ameerpet match too: the stored function self-joins each ride's path and requires pickup before drop-off."
4	M2	2:30	**Join this ride** on an instant ride, then on a full one (waitlist)	"The capacity trigger locks the ride row, so two people can never take the last seat. A full ride offers the waitlist; when someone cancels, a trigger promotes the first person who fits."
5	M2	3:30	**My rides → Hosting**: accept a request on Aarav's approval ride; then **Complete ride** on the departed ride to Hitech City	"Only the creator can decide (SQLSTATE 42501 otherwise). Start / complete follow the ride state machine; completing opens fare-splitting and ratings."
6	M3	4:40	**Record the fare** on that ride: ₹627, split by percent 50/30/20, then 60/60	"The preview and the stored shares come from the same function, `fn_expense_shares`. 60 + 60 ≠ 100 is refused (E15). Shares are in whole paise and always add up exactly: a deferred trigger re-checks Σ shares = amount at COMMIT."
7	M3	5:40	**Expenses**	"Balances with each friend come from `v_pairwise_balance`: every share is a debt edge, every confirmed payment an opposite edge."
8	M4	6:20	**Settle up → Pay** Aditya Singh for the airport ride (UPI, type a 5-digit UTR, then 12 digits)	"CHECK: UPI needs a 12-digit UTR. The payment is *pending* until the payee confirms; pending money is reserved, so the same debt can't be paid twice."
9	M4	7:10	**Net off** with Isha	"Aarav owes her on one ride and she owes him on another: one transaction writes two mirrored offset rows, and a deferred constraint trigger checks the pair at COMMIT. Rides are locked in ID order, so it can't deadlock."
10	M4	7:50	**Payments** → confirm the incoming ₹295; click **Remind** twice on Settle up	"The state machine is a table: payee confirms, payer cancels. The second reminder hits an EXCLUDE constraint over 24-hour time ranges."
11	M5	8:50	**Rate co-riders**: rate the airport ride's three co-riders, one of them 2★ without a comment	"The batch is one transaction: the 2★ needs a comment (CHECK), so nothing is saved. Fix it and all three go in; the audit trigger records each."
12	M5	9:50	**My profile** → trust network; **Leaderboard** → refresh	"Trust = Bayesian average + no-show rate. The network is a recursive CTE with a cycle guard. The leaderboard reads a materialized view refreshed CONCURRENTLY."
13	M5	10:50	**Campus dashboard**, **Rides & bookings**, **Money dashboard**	"Every chart is a named query; SQL shows the query, its concepts and EXPLAIN ANALYZE with the indexes used."
14	All	11:50	**Integrity Lab → Run all 81**	"Each scenario is raw SQL that bypasses the API. The database refuses every one by the mechanism we designed (CHECK, FK, PK, partial unique index, EXCLUDE, trigger, deferred trigger, stored function), then rolls back. Watch Member 4's two-session race: session B waits on A's row lock, re-reads and is refused."

#	Who	Time	Do this	Say this
15	All	13:30	**Database**	"The schema described by PostgreSQL itself: 25 tables, 138 constraints + 128 NOT NULL, 40 triggers, 47 functions, 19 views, 69 indexes, ~15,300 rows, and 13 cross-module invariants, all at zero."

psql one-liners

```
SELECT * FROM fn_integrity_report();                        -- 13 rows, all 0
SELECT * FROM v_student_overview WHERE student_id = '2023A7PS0001H';
SELECT * FROM fn_search_rides(1, 16, now(), NULL, 1, NULL, NULL) LIMIT 5;
SELECT * FROM payment_transition;                            -- the payment state machine as data
SELECT ride_id, sum(net) FROM v_ride_balance GROUP BY 1 HAVING sum(net) <> 0;   -- always empty
```

Questions every member should be able to answer

The ER and the whole schema

1. *Why 25 relations when the guideline says 8–12?* Five modules, 14 core entities/relationships; the other 11 are reference tables (normalization: no repeated strings) and append-only history tables written by triggers (auditability). Each one removes an anomaly; none duplicates data.
2. *What is the hub of the schema?* `ride_member (ride_id, student_id)`. Expenses, payments, reminders and ratings reference it with **composite** foreign keys, so "only people on this ride" is declarative everywhere.
3. *Which are weak entities?* `ride_stop` (identified by ride + sequence) and `expense_share` (by expense + student); `rating_tag_map` is an associative entity.
4. *Is anything stored that could be derived?* No balance, seat count, fare per head or trust score is stored. The one controlled redundancy is the materialized view `mv_student_reputation`, refreshed on demand; `distance_km` is a fact of the ride fixed at creation.
5. *Why put rules in the database instead of the API?* One enforcer for every client (API, psql, the seed, future apps). The Integrity Lab bypasses the API on purpose to prove it.
6. *How do triggers report errors?* `RAISE … USING ERRCODE = '23514', CONSTRAINT = 'rule_…'`, the same SQLSTATE as a CHECK, so the API maps both through one rule catalogue (123 rules, `server/src/rules/`).
7. *Normal form?* BCNF for every relation; each determinant is a candidate key (e.g. `student`: id, email and phone are all keys; `payment`: id, client_ref, utr).
8. *How did you integrate?* Shared contract of names; stand-ins deleted; Member 3's settlement view re-pointed to confirmed payments; duplicate indexes removed; one seed pipeline; one login. See `INTEGRATION_NOTES.md`.

Member 1: users, rides, search

- *How is a BITS ID validated?* CHECK with a regex; a trigger normalises case and spaces first.
- *How does "no overlapping rides" survive two simultaneous requests?* `fn_create_ride` locks the student's row (`FOR UPDATE`), so the second request waits and then sees the first ride. A test fires four at once: exactly one wins.
- *How does search find a ride via a stop?* Build each ride's path (origin 0, stops 1–3, destination 99) with UNION ALL, self-join it, require pickup seq < drop-off seq.
- *Ride state machine?* open ⇄ full → departed → completed, open/full → cancelled; any other change is refused by a trigger and every change is logged in `ride_status_log`.

Member 2: booking, pooling, seats

- *How can't a ride be overbooked?* The capacity trigger locks the ride row and sums confirmed seats; concurrency test: 6 people for 2 seats, exactly 2 win.
- *How does the waitlist work?* FIFO by `joined_at`, first-fit: when seats free up, a trigger walks the queue and books every entry that fits; position is `ROW_NUMBER() OVER`.
- *One pending request per ride?* A **partial unique index** `(ride_id, student_id) WHERE status = 'pending'`.
- *Why is booking_event append-only?* It is the notification feed and audit trail; an UPDATE trigger refuses changes.

Member 3: expense splitting

- *How are shares rounded?* Largest-remainder in paise: floor every share, give the leftover paise to the largest fractions; ₹100 / 3 = 33.34 + 33.33 + 33.33.
- *Why a deferred trigger?* Shares are inserted one row at a time; an immediate check would fail after the first row. Deferred, it runs at COMMIT when all shares exist.
- *Why can't an expense be edited?* Amount, payer and split are frozen by a trigger (delete and re-add instead), and deletes are blocked once money moved on the ride.
- *How does M3 see payments?* Only through `v_settlement_flow` = confirmed payments.

Member 4: payments and settlement

- *Why is there no balance column?* It would be derived data with an update anomaly in three modules; `v_ride_balance` derives it.
- *How is double payment prevented?* The insert trigger locks the ride row and counts pending payments as reserved; three concurrent payments → exactly one succeeds.
- *Idempotency?* A client UUID (`client_ref`, UNIQUE); a retry returns the same payment.
- *EXCLUDE vs UNIQUE?* EXCLUDE generalises UNIQUE to any operators: `=` on ride/creditor/debtor plus `&&` (overlap) on a 24-hour range.

Member 5: ratings and analytics

- *How is the trust score computed?* Bayesian average (shrinks small samples to the campus mean) weighted 80%, reliability (1 − no-show rate) 20%.
- *Why a materialized view?* The leaderboard ranks every student; the MV makes it one index scan, refreshed CONCURRENTLY (needs a unique index).
- *What does the recursive CTE do?* Walks "A rated B ≥ 4.5 on average" edges up to 3 hops, blocking cycles with a path array and keeping the shortest path (DISTINCT ON).
- *Why can't you rate a no-show?* The FK passes (they are a member) but the trigger checks both statuses are `confirmed`.

Deeper, module-by-module answers are in `docs/modules/` (each member's own design and viva notes).